

WardWatch Design Documents

Class Diagrams · Sequence Diagrams · Wireframes

COMS3009A Software Design 2026 · Wits University

This document contains the design-level artefacts for WardWatch: a class diagram showing the Firestore data model and service layer, a sequence diagram tracing the worker claim-and-update flow (US023/US024), wireframes for the three key authenticated dashboards, and a table of key design decisions. These supplement the architecture diagrams submitted separately.

1. Class Diagram — Firestore Data Model & Service Layer

The class diagram shows the six Firestore collections, their fields, relationships between them, and the three frontend service files that wrap the Firebase SDK calls.

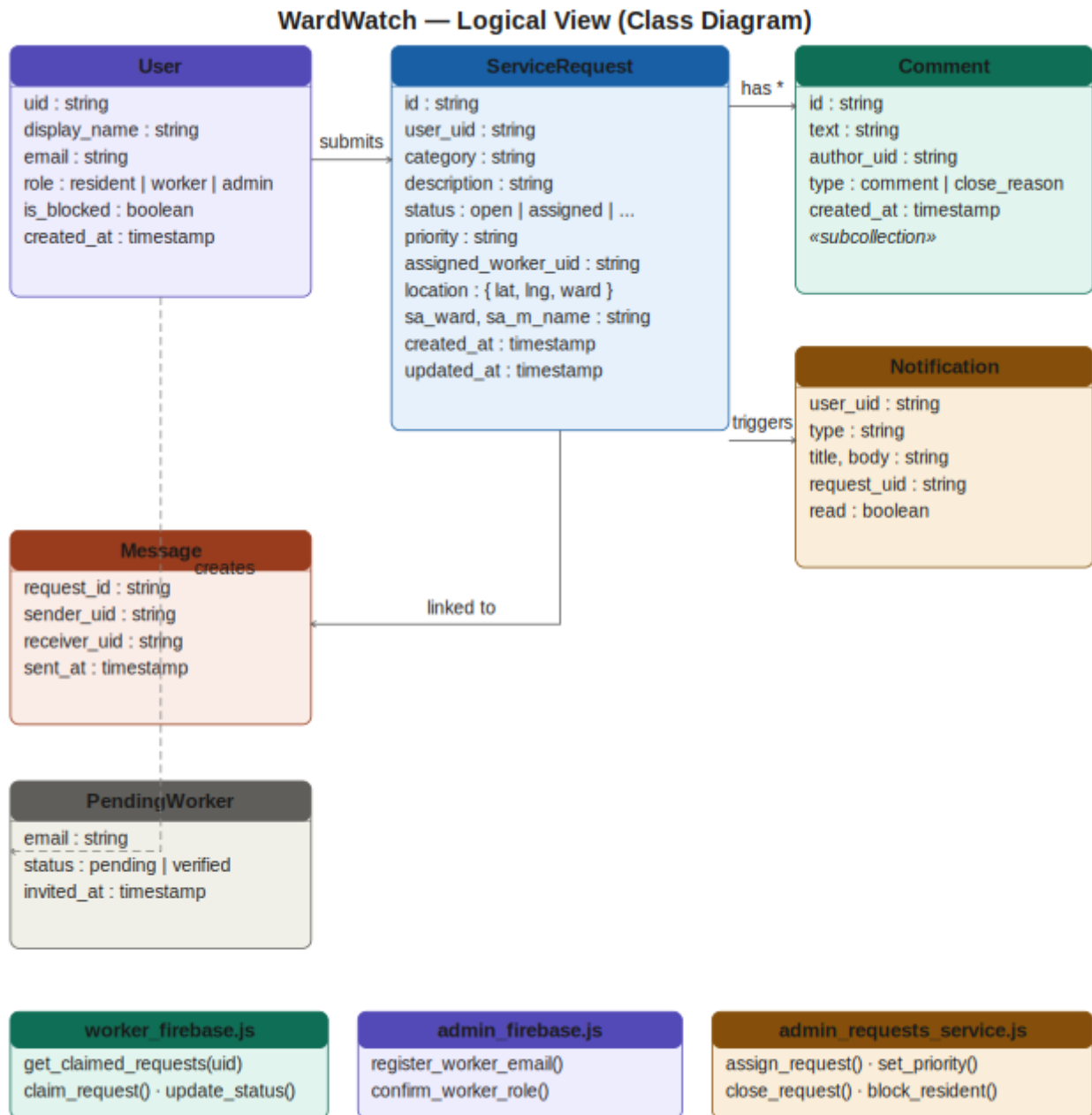


Figure 1 — Class diagram. Collections: User, ServiceRequest, Comment (subcollection), Notification, Message, PendingWorker. Service layer shown below.

1.1 Key Relationships

Relationship	Multiplicity	Description
User → ServiceRequest	1 to many	A resident submits many requests; user_uid stored on each
ServiceRequest → Comment	1 to many	Comments are a subcollection under each request document
ServiceRequest → Notification	1 to many	Status changes trigger notifications to resident and worker
ServiceRequest → Message	1 to many	Private messages linked to a request via request_id
Admin → PendingWorker	1 to many	Admin registers worker emails; stored until verified

2. Sequence Diagram — Worker Claims and Updates a Request

Traces the runtime interaction for US023 (claim_request) and US024 (update_request_status). Each status change calls notify_status_change(), which writes notifications to both the resident and worker atomically using a Firestore writeBatch.

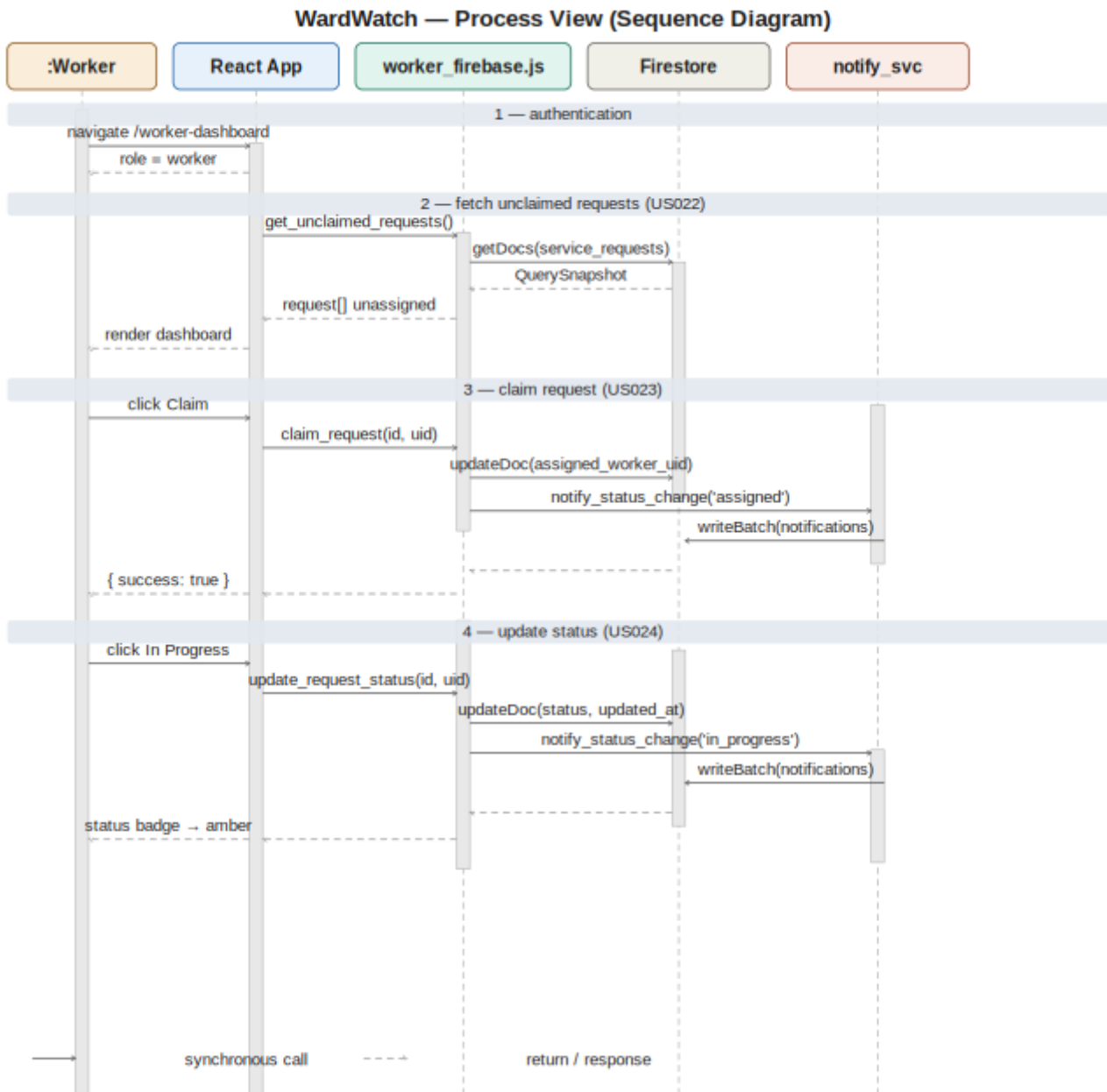


Figure 2 — Sequence diagram: Worker → React App → worker_firebase.js → Firestore → notify_svc. Covers US023 and US024.

2.1 Participants

Participant	Type	File / Location
:Worker	Actor	Browser — authenticated Firebase user with role=worker
React App	Component	src/pages/worker_dashboard/worker_dashboard.js
worker_firebase.js	Component	src/backend/worker_firebase.js
Firestore	External	Google Cloud Firestore — service_requests collection

notify_svc	Component	notify_status_change() in worker_firebase.js
------------	-----------	--

3. Wireframes — Key Authenticated Screens

Low-fidelity wireframes showing component placement, navigation, and key interactive elements for the three primary authenticated views. Navy/Amber civic design system throughout.

3.1 Worker Dashboard (/worker-dashboard)

Displays assigned and unassigned request queues. Selecting a request opens a slide-in detail panel (RequestDetailPanel) with status buttons and comment history.

WardWatch [worker_nav_bar] Hello, {name} ■ Notifications Messages
[My Assigned Requests] [Unassigned Requests]
Filter: Status ■ Ward ■ Priority ■ Search...
#001 · Pothole · Ward 12 · HIGH Status: assigned · Submitted 2 days ago [Update Status ■] [View Detail →]
#002 · Water outage · Ward 8 · MEDIUM Status: in_progress · Submitted 5 days ago [Update Status ■] [View Detail →]
[< Prev] Page 1 of 3 [Next >]

Figure 3 — Worker dashboard. Tab row selects queue; each card has inline status control.

3.2 Admin Dashboard (/admin/requests)

Full system overview with sidebar navigation. Main panel lists all requests with inline assign, priority, and close-with-comment controls.

WardWatch Admin Panel

Workers ■■■■■■
Requests ■
Messages
Categories
Analytics

All Requests Filter: Status ■ Worker ■ Priority ■ Ward ■ ID · Category · Ward
Status · Priority · Worker · Actions #001 · Pothole · W12 · assigned · HIGH · —
[Assign ■][Priority ■] #002 · Water · W8 · in_progress · MEDIUM · J. Smith ·
[Assign ■][Priority ■] #003 · Electric · W5 · submitted · URGENT · — · [Assign
■][Priority ■] #004 · Waste · W3 · resolved · LOW · T. Dlamini · [Close w/
comment]

Figure 4 — Admin dashboard. Sidebar navigation; main panel with assign, priority, close controls.

3.3 Resident Dashboard (/resident-dashboard)

Authenticated residents see their request list, a submit button, stat summary, and options to cancel unassigned requests or rate resolved ones.

WardWatch [nav_bar] My Dashboard Hello, {name} ■
[+ Submit New Request]
Total: 4 · Open: 2 · In Progress: 1 · Resolved: 1
MY REQUESTS

#001 · Pothole on Main St · Ward 12 · IN PROGRESS · High Worker: J. Smith · Last updated: 1 day ago [View Detail & Comments →]
#002 · Water outage · Ward 12 · SUBMITTED · Medium Unassigned · Submitted: 3 days ago [View Detail →] [Cancel Request]
#003 · Street light · Ward 12 · RESOLVED · Low Resolved by T. Dlamini · 4 days ago [★ Leave Feedback]

Figure 5 — Resident dashboard. Stats row, own requests list, cancel option (unassigned), feedback prompt (resolved).

4. Key Design Decisions

Decision	Rationale
Comments as Firestore subcollection	Co-located with request; efficient to query; avoids cross-collection joins
Slide-in RequestDetailPanel (no separate <code>Post</code>)	Preserves dashboard context; faster UX for queue management
<code>writeBatch</code> in <code>notify_status_change()</code>	Atomic write: both resident + worker notifications created together or not at all
One Firebase service file per actor role	Clear separation of concerns; each file only imports what its actor needs; easier to test
Separate <code>public_dashboard_service.js</code>	Different data shape (normalised + filtered); keeps unauthenticated reads separate
StatsSA GeoJSON auto-tagging on submit	Residents should not need to know their ward number; ensures accuracy and consistency
Express <code>server.js</code> alongside React CRA	Handles S3 photo uploads with pre-signed URLs and Firebase Admin SDK for privileged ops